

Timer, Counter and PWM

Boya Vinay Kumar
Vijaykumar U

Why Timers Matter: Real life examples

1)Coffee vending machine

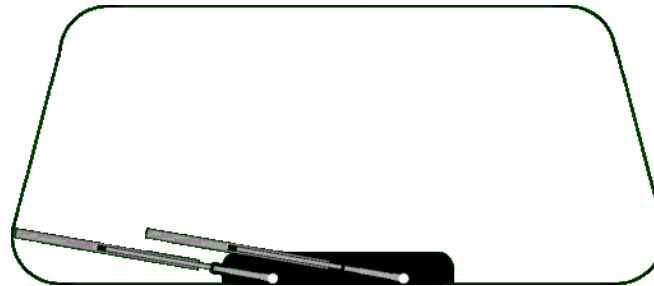
2)Washing machine

3)Traffic signal

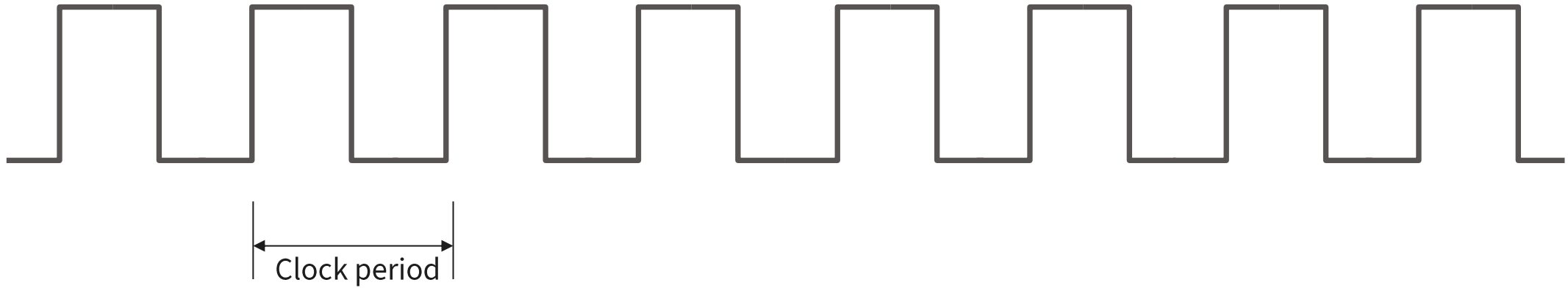
4)Car wiper

5)Induction stove

6)Refrigerator door open warning



Timekeeping on microcontrollers – 1/2



$$\text{Clock Period } T = 1/\text{Frequency}$$

Example:

Frequency: 100 Hz

Period: $1/100 = 0.01\text{s}$ or 10ms

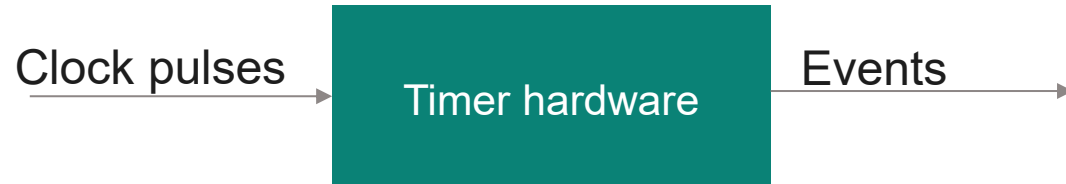
Timekeeping on microcontrollers – 2/2



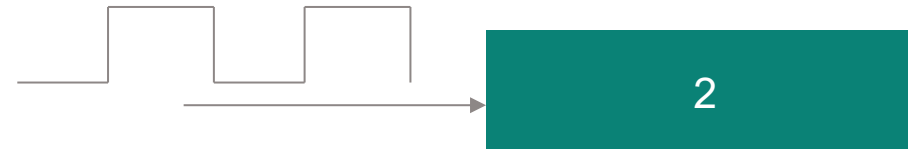
Quiz: If each clock period is 10ms, what is the overall duration of 200 clock periods?

Answer: 2s

Introduction of Timers and Counters



1 clock pulse given to timer hardware
Timer counts 1



2 clock pulse given to timer hardware
Timer counts 2

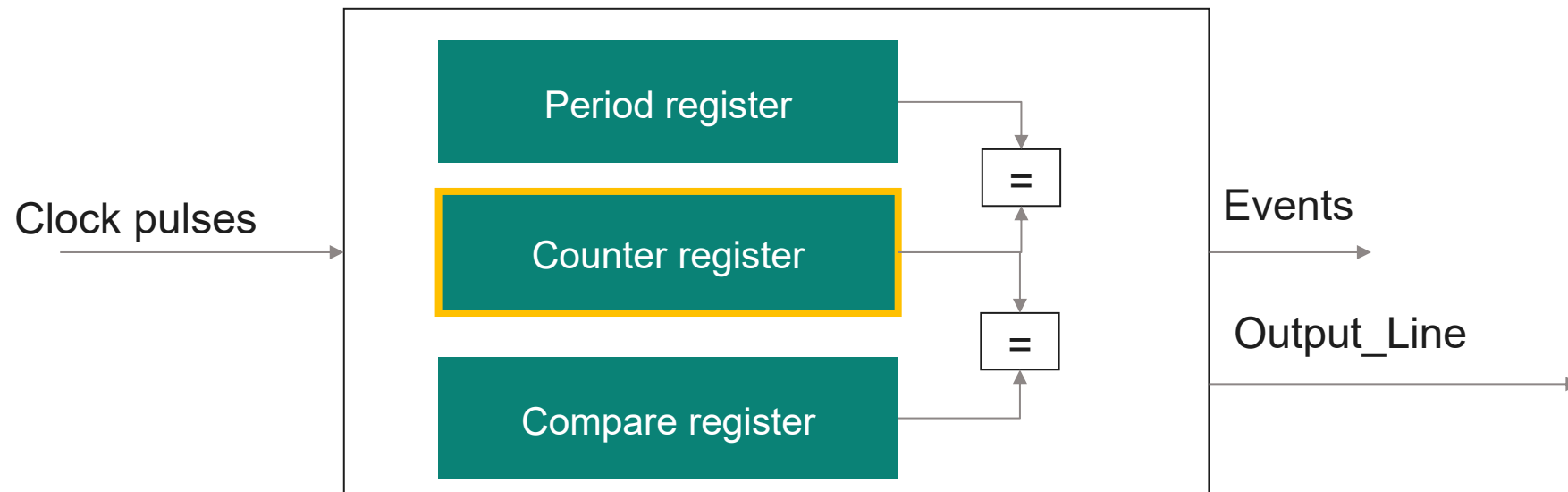
Example

Timer has counted up to 5.

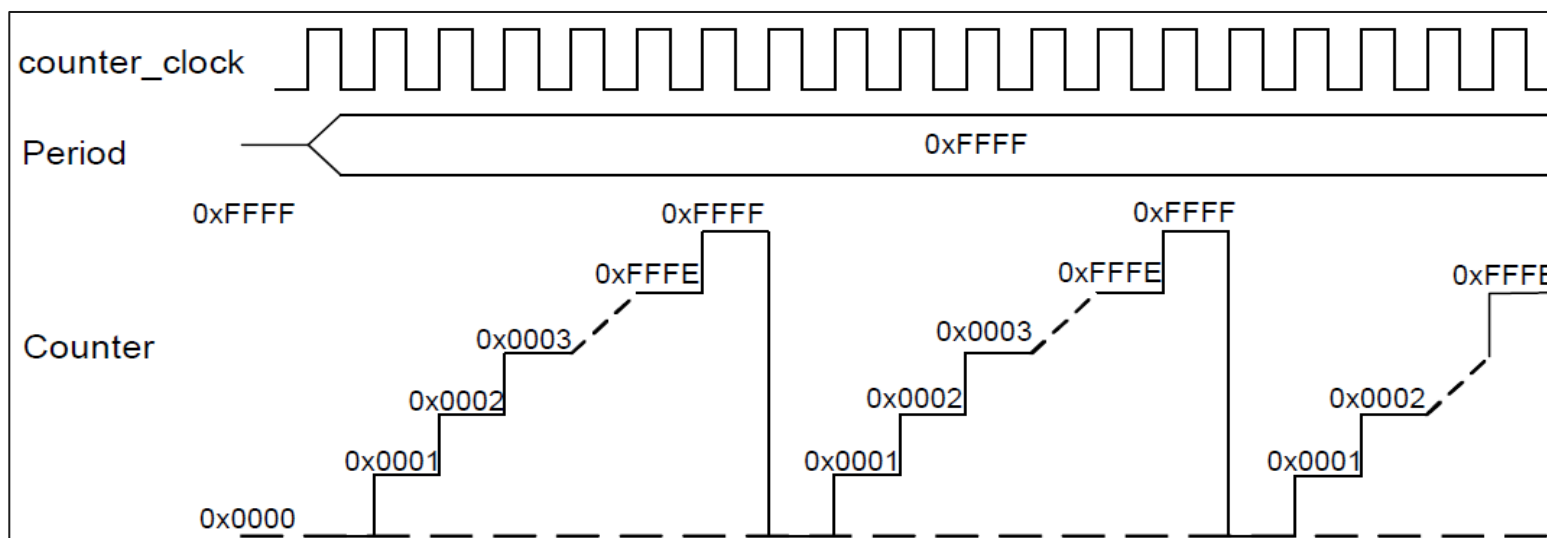
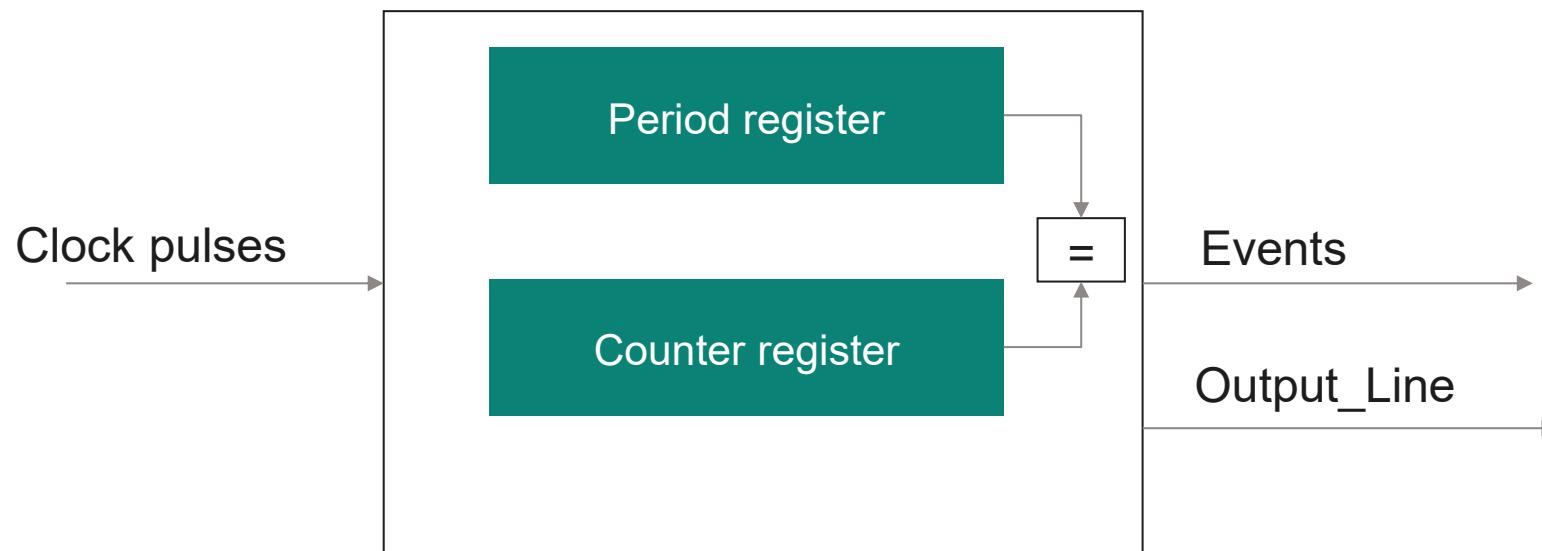
It means the timer hardware has received 5 clock pulses.

If each clock pulse is of period 0.5 S, we may say that $5 \times 0.5 \text{ S} = 2.5 \text{ S}$ have lapsed

General architecture of hardware timer



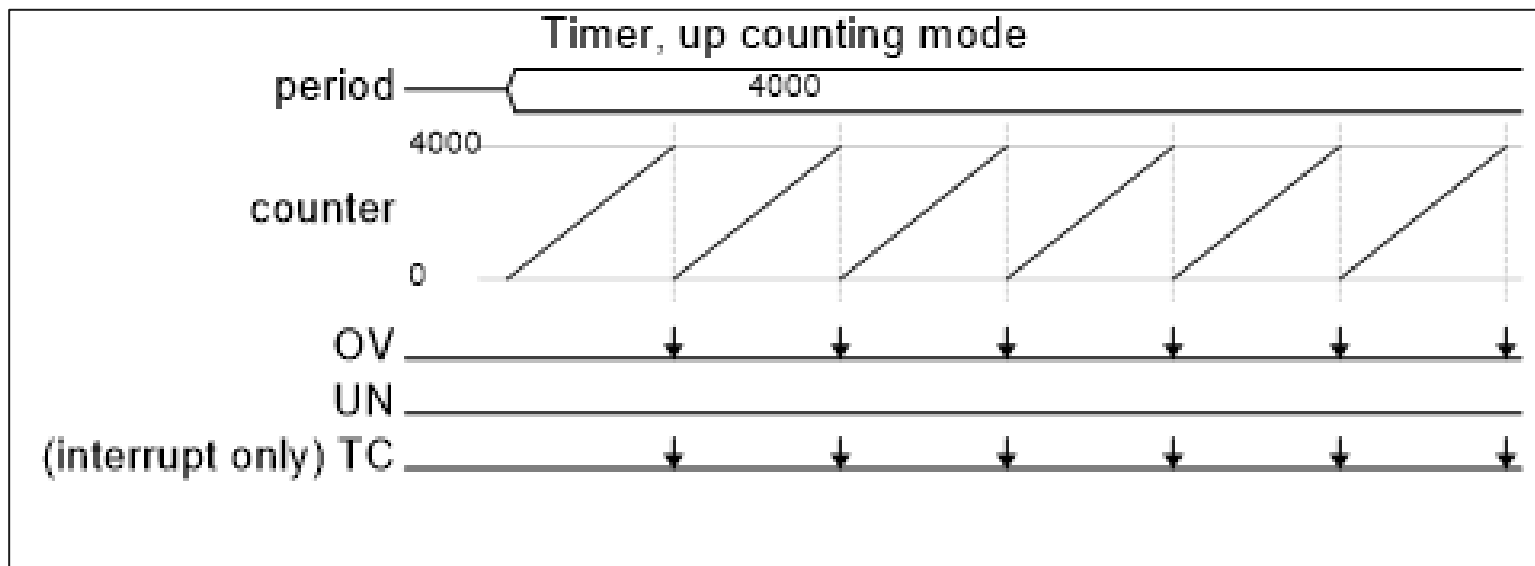
Hardware timer as a simple timer



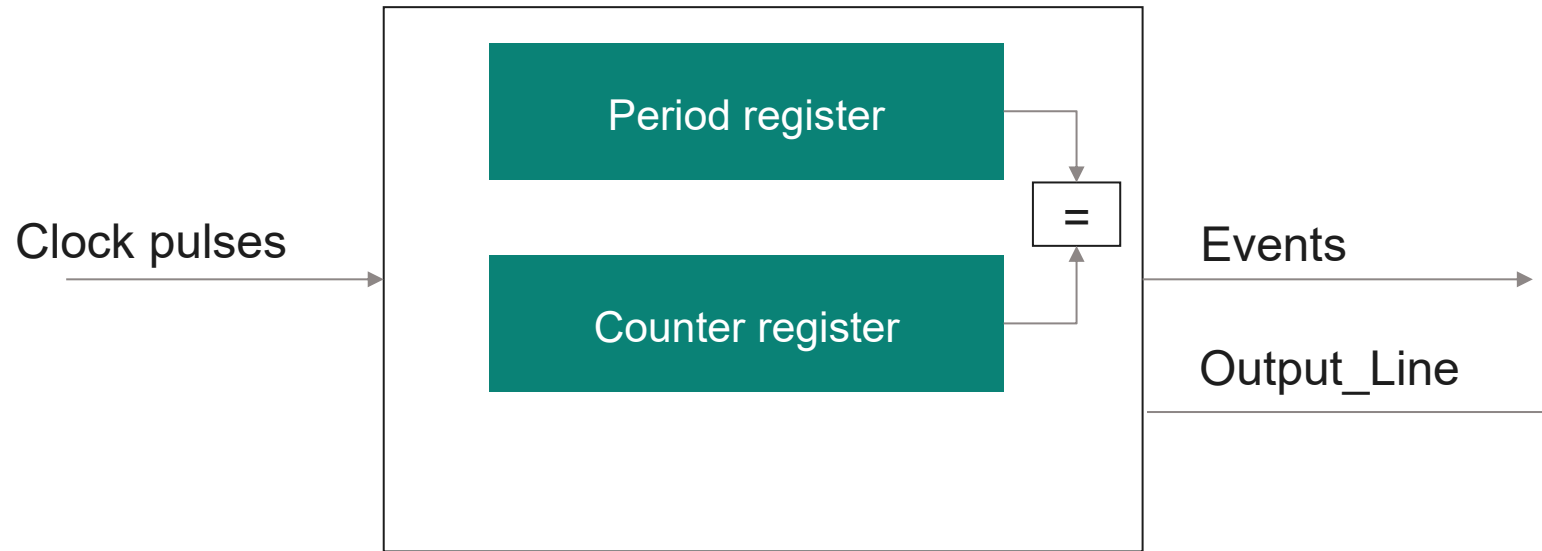
Timer counting up from 0 to terminal count

Example:

1. Your application has a need to wait for exactly 2 Sec before doing something else.
2. You know that the clock to the timer hardware is 2000 Hz and therefore the clock period is 0.5 milli-second
3. You program the Period register with a value of 4000 (Terminal Count Value) and START the timer
4. The counter register which was initialized to 0 increments by 1 for every clock pulse
5. After reaching a value of 4000, the Counter and Period registers are equal.
6. Timer hardware generates an event. Before starting the timer, you configured this event to interrupt the CPU
7. If the Interrupt Service Routine executes, it will mean that the wait time of 2 Sec has lapsed.



Hardware timer as a code profiler



You want to find out how long does a piece of software take to execute.

For example, you wrote FFT algorithm and now you want to know how long it takes to fully execute.

1. Load the Period register with the highest value (e.g. 0xFFFFFFFF).
2. Start the timer.
3. Run the FFT code
4. Now stop the timer and make a note of the value in the counter register
5. Let us assume that you read out a value of 12780 from the counter register.
6. If the clock frequency was 2000 Hz, then the 12780 clock pulses total to 6390 ms or 6.39 Sec

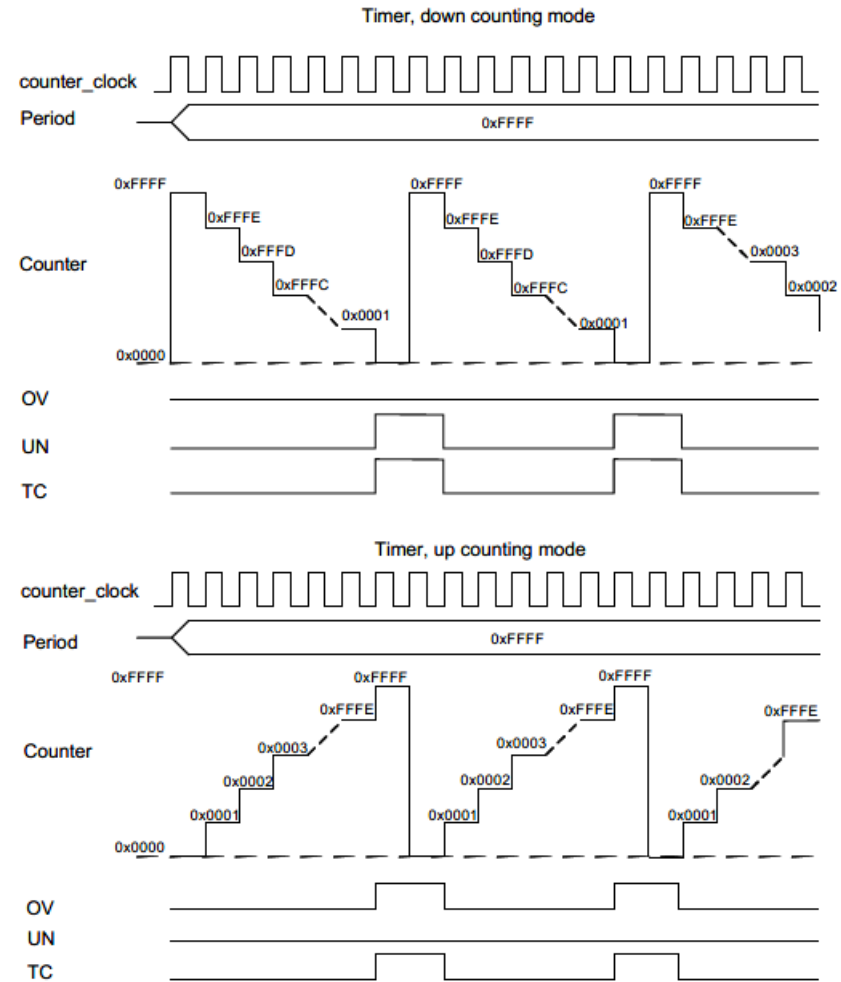
Applications of Timer/counter

Precision Timing Operations	Timer modules ensure accurate time management for various applications, enhancing system reliability.
Delay Generation	They generate precise delays required for synchronizing processes within microcontroller applications.
Event Counting	They can count specific events, enabling monitoring and control of various system parameters.
Input Capture	Timers can capture the timestamp of an external event. This is useful for measuring the time interval between events, such as the period of a waveform or the duration of a pulse.
Output Compare	Timers can be configured to generate an output signal when the timer reaches a specific count value. This can be used to generate precise periodic signals, trigger interrupts, or control timing for other tasks.
Frequency Measurement	Timers can be used to measure the frequency of an input signal by counting the number of events in a fixed time period. This is useful in applications like frequency counters and signal analysis.
Watchdog Timer	A special type of timer, the watchdog timer, is used to reset the microcontroller if the software fails to operate correctly. This helps in recovering from software malfunctions and ensures system reliability.
Task Scheduling	In real-time operating systems (RTOS), timers are used to schedule tasks to run at specific intervals, manage time slices in multitasking systems, and ensure that time-critical tasks are executed as needed.

Introduction to Timer module - TCPWM in PSoC

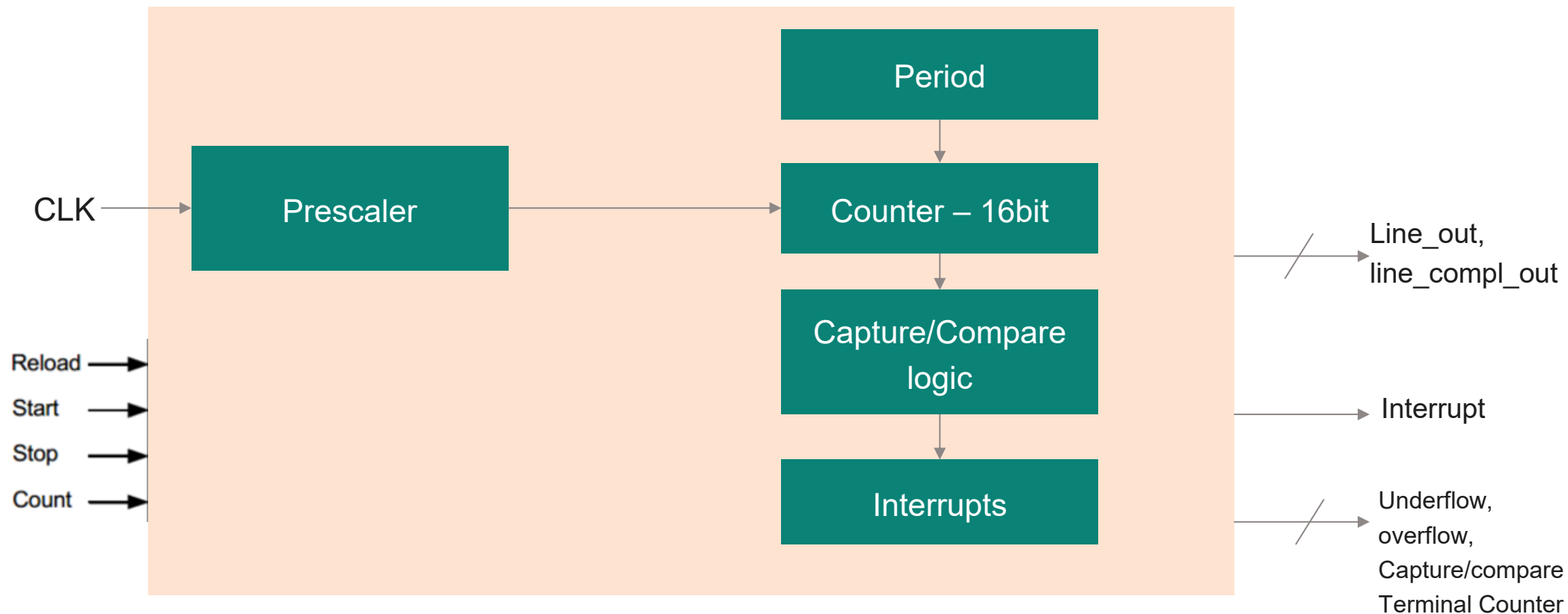
The block can be used to measure the

- Period and pulse width of an input signal (Timer),
- find the number of times a particular event occurs (Counter),
- generate PWM signals

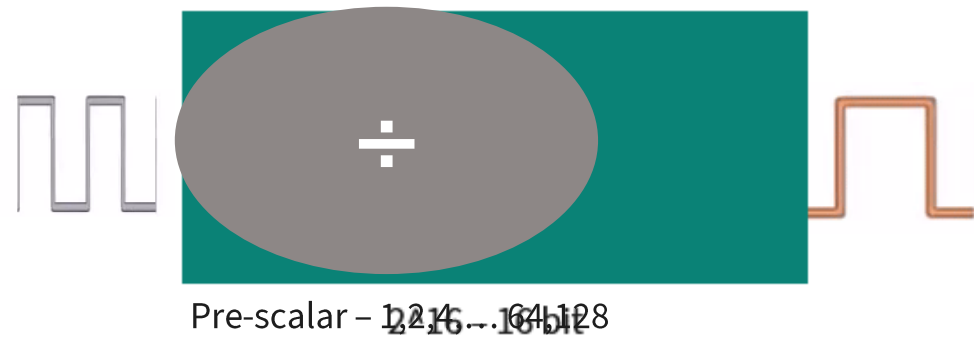


Key Components of Timer module

- Prescaler - Divides the input clock frequency to extend the timer range.
- Counter - Registers the number of clock cycles for accurate timing.
- Capture/Compare Units - Captures external events or compares timer values to trigger actions.
- Interrupts - Allow timers to signal the CPU asynchronously for timely responses.

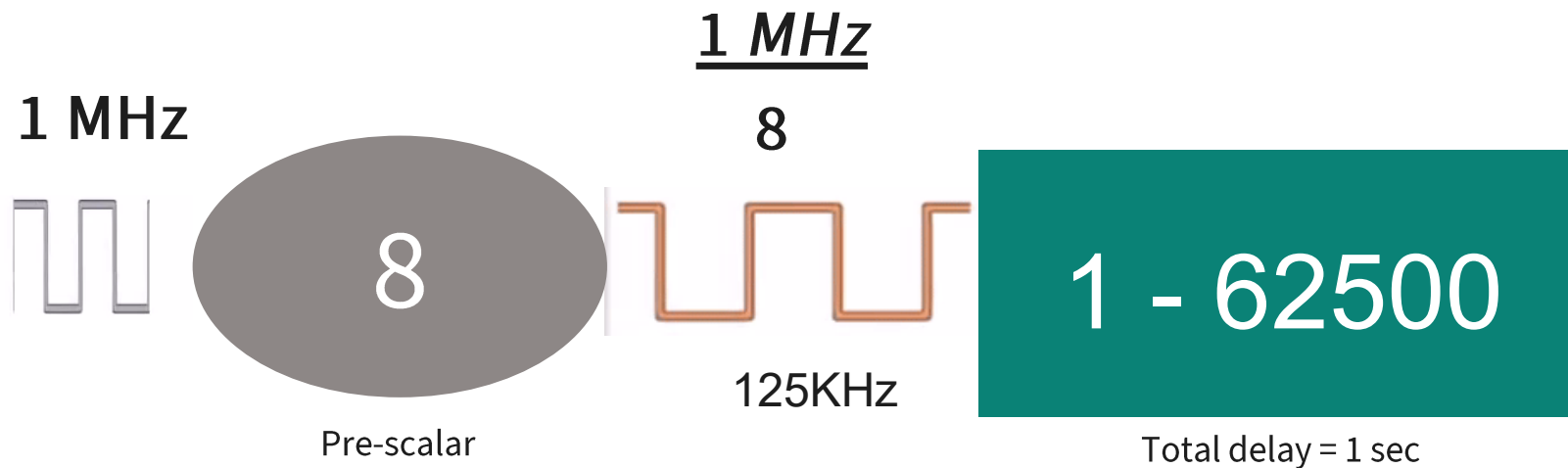


Pre-scalar



Motivation for pre-scalar

- If you need to measure lot of time (more seconds) with an 8-bit timer that overflows at 256, you can use a pre-scalar to give your timer a slow clock for desired requirement.
- For shorter (8 and 16-bit) timers, there will often be a tradeoff between resolution (high resolution requires a high clock rate) and range (high clock rates cause the timer to overflow more quickly).



Without Pre-scaler:

$$\text{Delay} = \text{Count} * (1/1\text{MHz})$$

where Count = 62500

$$\text{Delay} = 0.0625$$

$$\text{Resolution} = 1/1\text{MHz} = 1\mu\text{S}$$

With 8 as Pre-scaler:

$$\text{Delay} = \text{Count} * (8/1\text{MHz})$$

where: Count = 62500

$$\text{Delay} = 0.5\text{seconds}$$

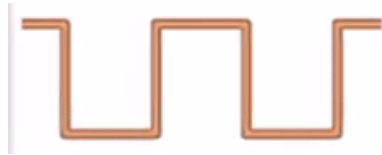
$$\text{Resolution} = 1/125\text{KHz} = 8\mu\text{S}$$

What about Resolution? will it change with Pre-scaler?

COUNTER

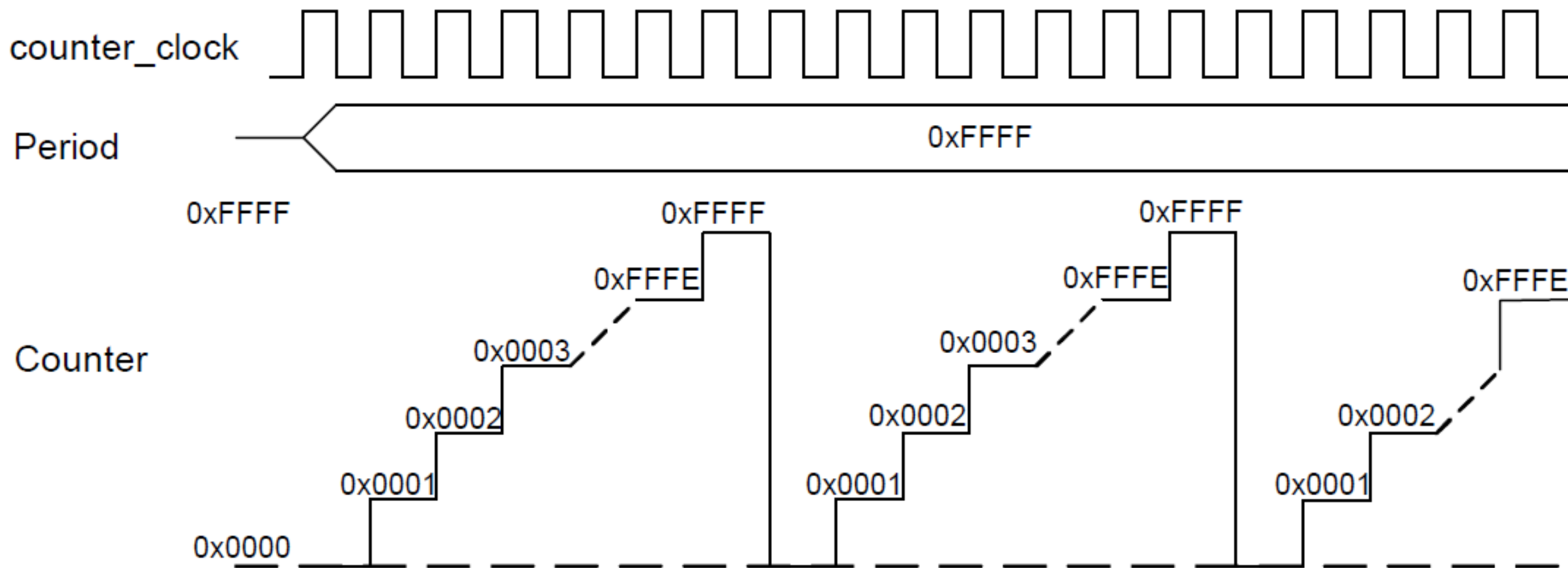
Modes of counter

Up Counter



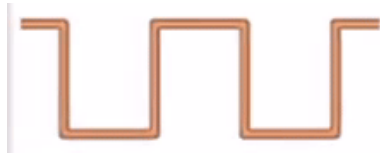
Period = 9

When Count = Period value it resets to 0



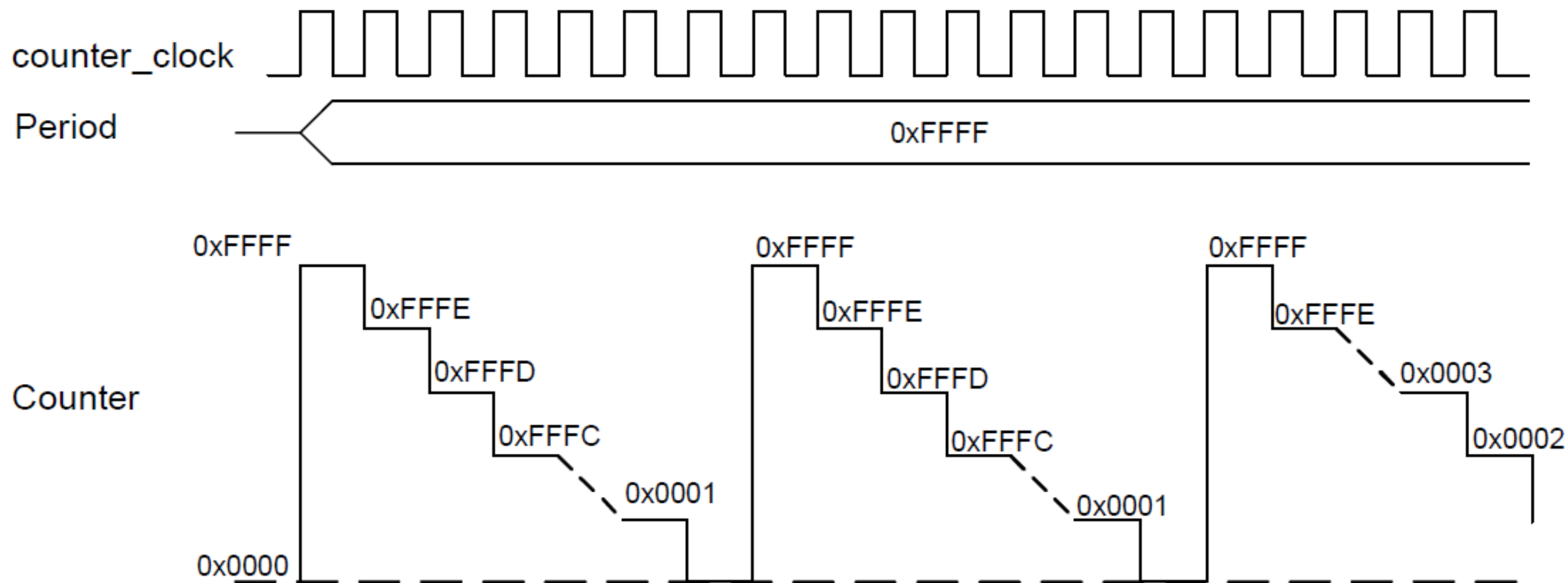
Modes of counter

Down Counter



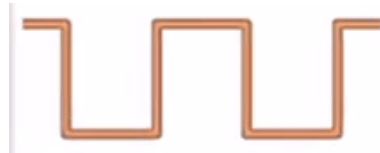
Period = 9

When Count = 0 value it resets to Period value



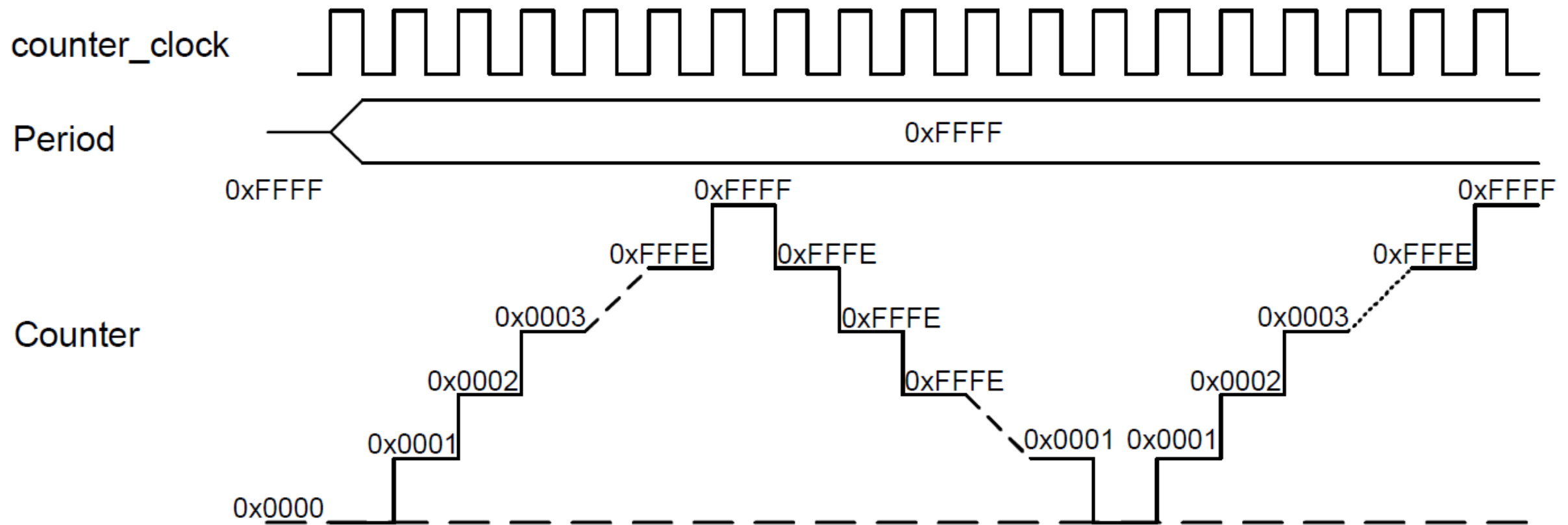
Modes of counter

Up/Down Counter



Period = 9

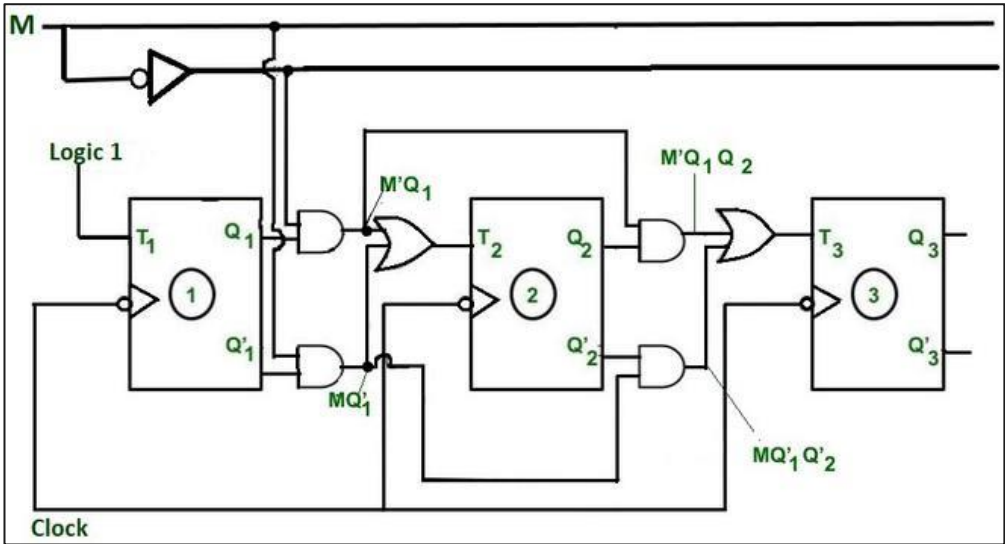
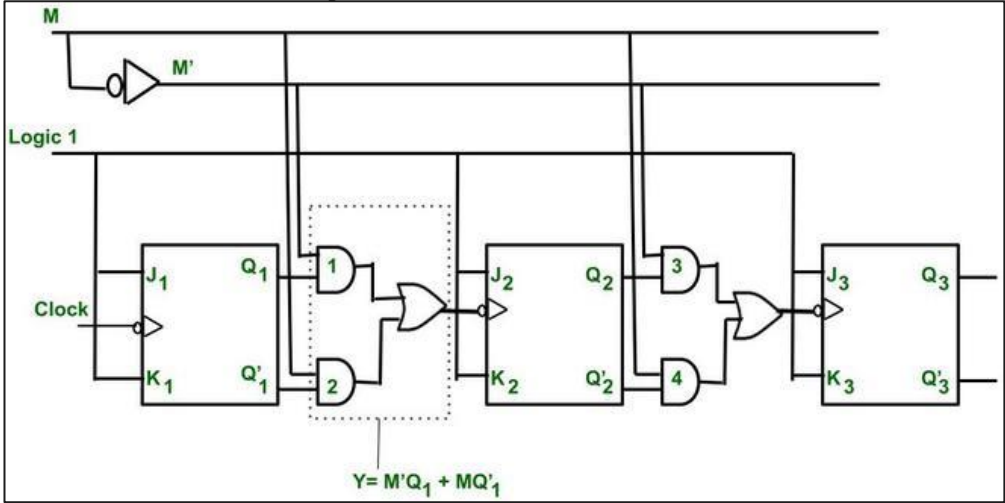
When Count = Period value it decrements to 0 and then again it starts incrementing till it reaches Period values and continues like this.



Example Counter Implementation: FLIP FLOP UP/DOWN COUNTER (JK for Async, T for Sync)

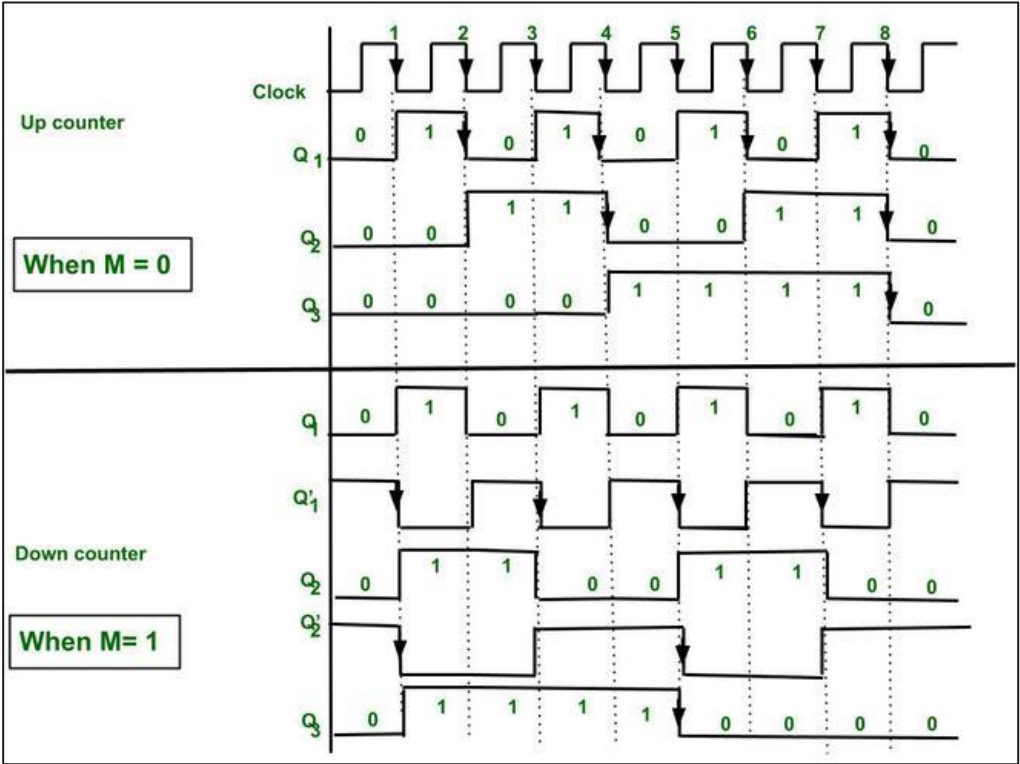


Asynchronous



synchronous

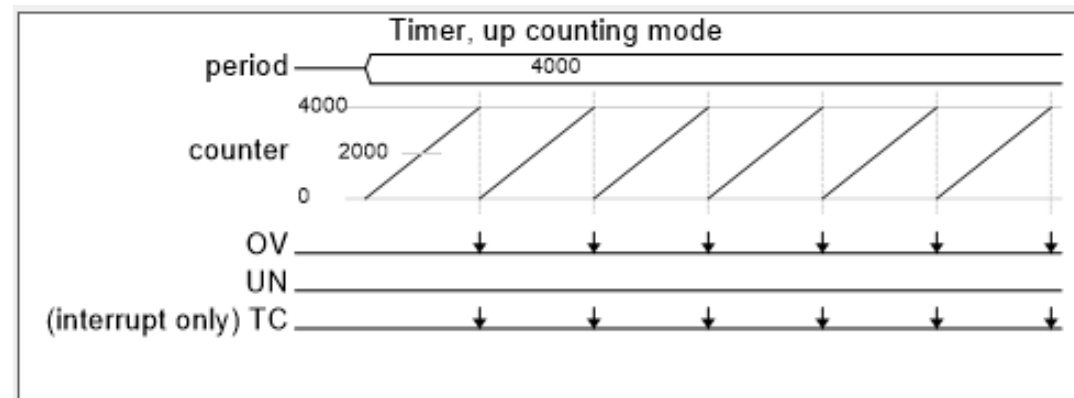
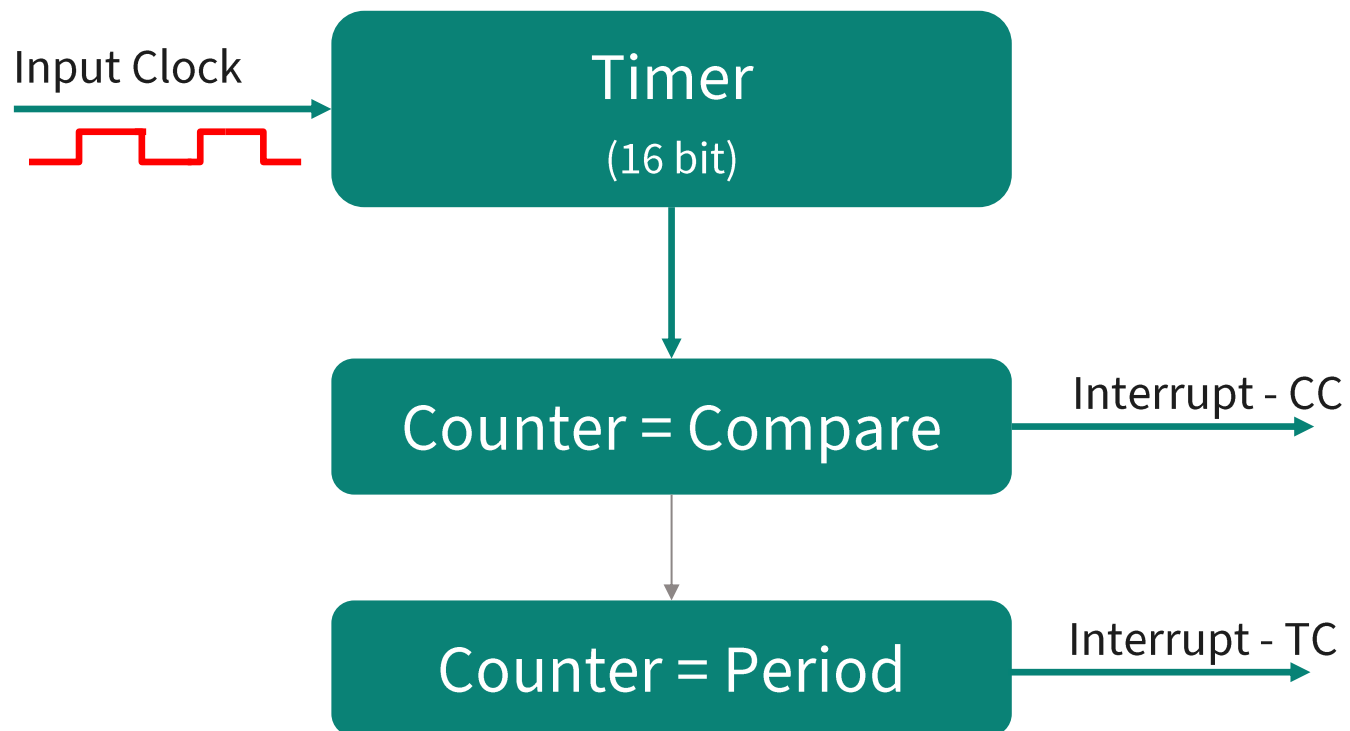
Timing Diagram



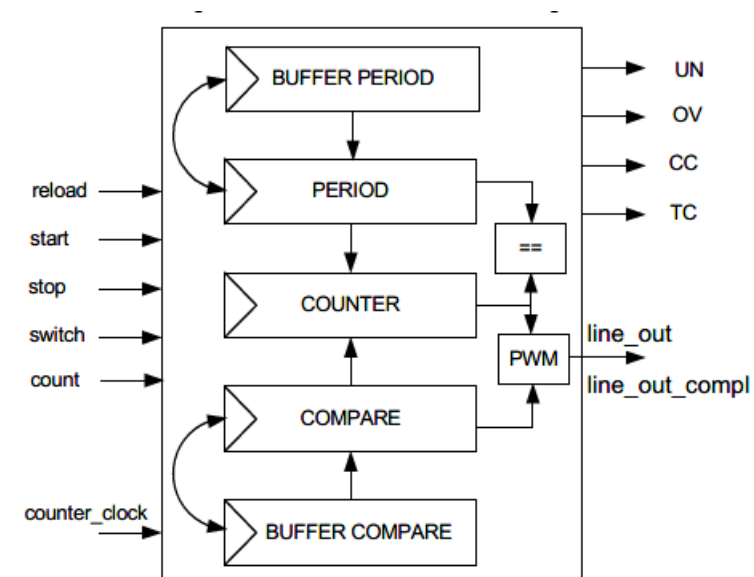
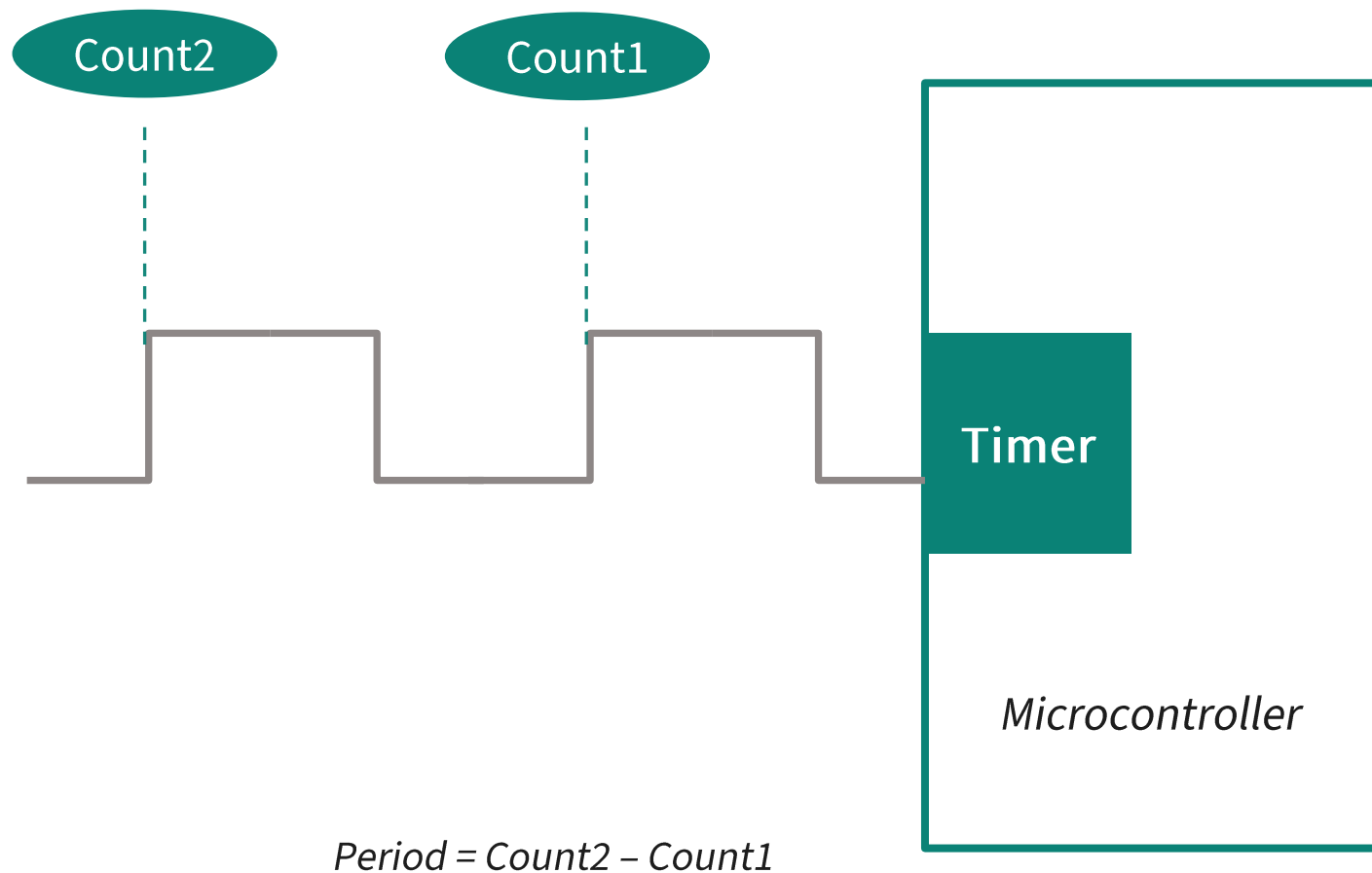
There are two types of counter mode

- Compare Mode
- Capture Mode

Compare mode

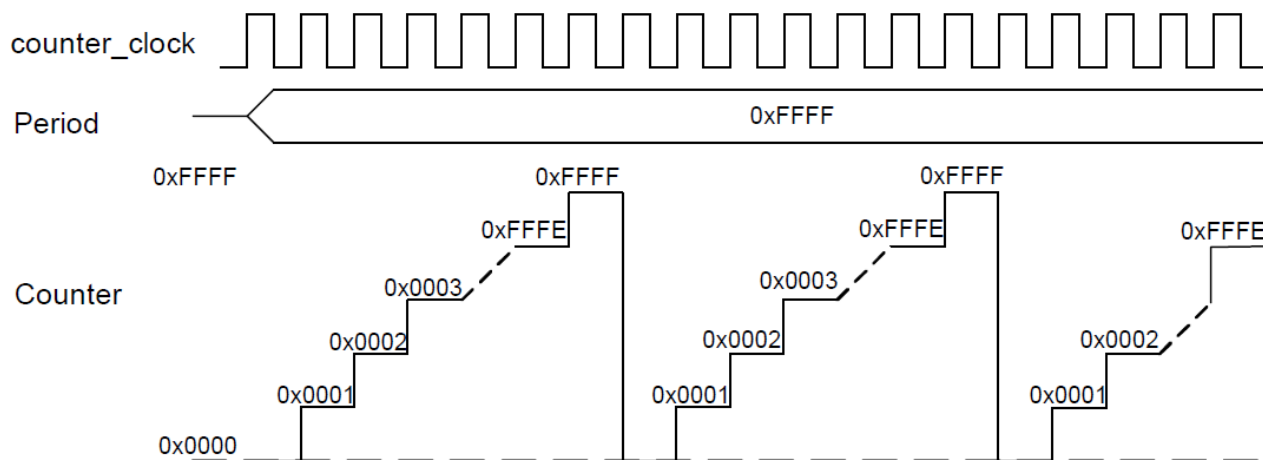
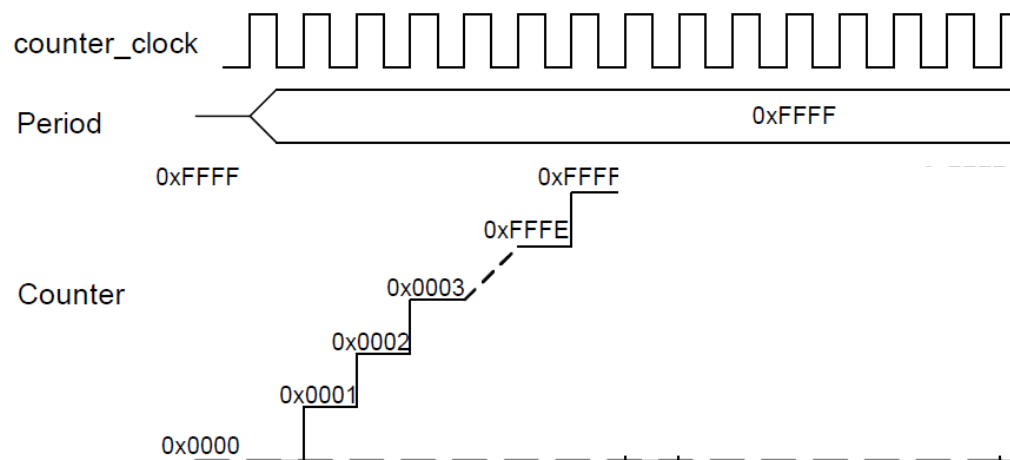


Capture mode



Hardware timer: Single shot vs Continuous modes

- A timer can count up to the PERIOD register value and two things can happen now
 - The counter can reset to 0 and start incrementing again [Up-counting, Continuous Mode]
 - The Counter can reload itself with value from PERIOD register and start decrementing again [Down-Counting, Continuous mode]
 - Or the timer simply stops [Single shot mode]



Register configuration

PSoC timer registers

Common Timer Registers

Register Name	Address
TCPWM_CTRL	0x40200000
TCPWM_CMD	0x40200008
TCPWM_INTR_CAUSE	0x4020000C

Registers of Timer-0

Registers of Timer-1

...

Registers of Timer-7

TCPWM_CNT7_CTRL	0x402002C0
TCPWM_CNT7_STATUS	0x402002C4
TCPWM_CNT7_COUNTER	0x402002C8
TCPWM_CNT7_CC	0x402002CC
TCPWM_CNT7_CC_BUFF	0x402002D0
TCPWM_CNT7_PERIOD	0x402002D4
TCPWM_CNT7_PERIOD_BUFF	0x402002D8
TCPWM_CNT7_TR_CTRL0	0x402002E0
TCPWM_CNT7_TR_CTRL1	0x402002E4
TCPWM_CNT7_TR_CTRL2	0x402002E8
TCPWM_CNT7_INTR	0x402002F0
TCPWM_CNT7_INTR_SET	0x402002F4
TCPWM_CNT7_INTR_MASK	0x402002F8

Understanding PSoC Timer registers – 1/2

Common registers

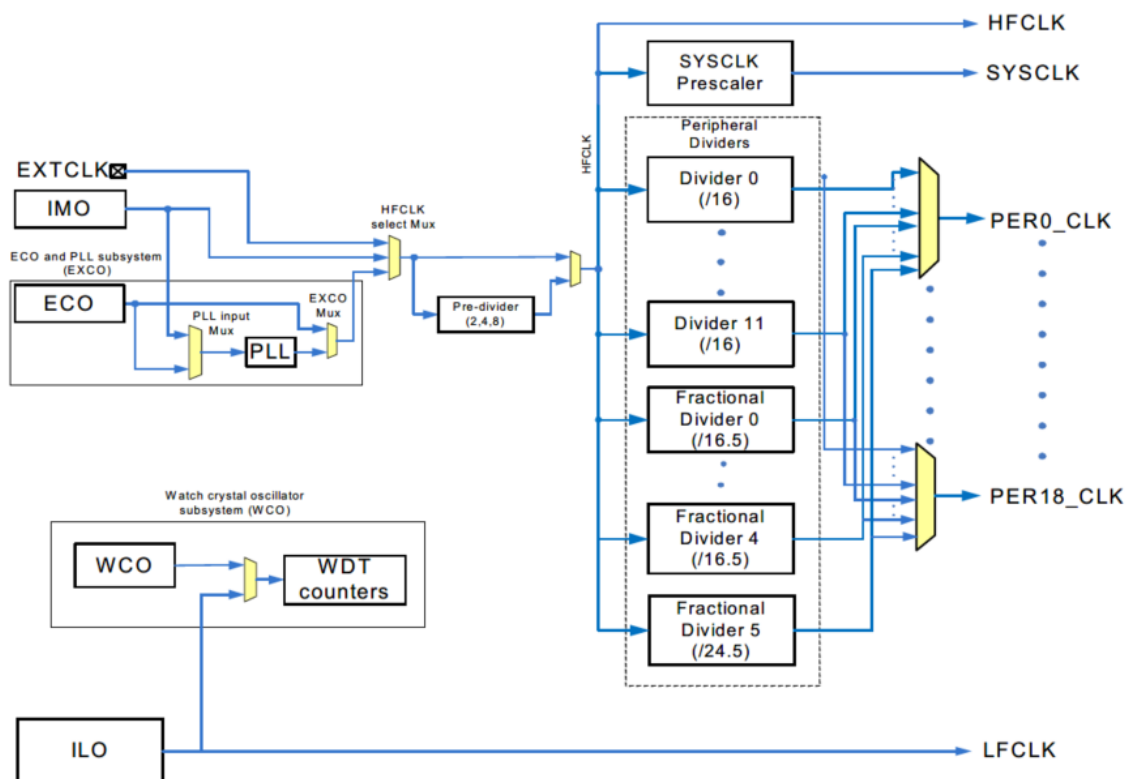
Register Name	Description
TCPWM_CTRL	Contains one Enable bit for each of the 8 timers. Your chosen timer will simply not work without activating the corresponding Enable bit here in this register
TCPWM_CMD	Contains the activation bits for generating control signals (START, STOP, RELOAD) to your chosen timer “by software”. If you prefer that other peripherals generate these control signals instead of software, use the timer specific register set
TCPWM_INTR_CAUSE	Contains one interrupt status bit for each of the 8 timers. If the bit corresponding to your chosen timer is found to be a 1 here, it means that your timer has generated an event (overflow, underflow, compare match, period match etc.) and that event has generated an interrupt to the CPU

Understanding PSoC Timer registers – 2/2

Timer specific registers

Register Name	Description
TCPWM_CNTx_CTRL	Contains bits that define operational mode of the timer (Timer/PWM mode, One-shot/Continuous, Up/Down/Up-Down mode, Reload from Shadow register)
TCPWM_CNTx_STATUS	1 bit to indicate whether the timer is running or not
TCPWM_CNTx_CC, TCPWM_CNTx_PERIOD, TCPWM_CNTx_COUNTER	COMPARE, PERIOD and COUNTER registers of this timer. Their shadow registers are TCPWM_CNTx_CC_Buff and TCPWM_CNTx_PERIOD_Buff
TCPWM_CNTx_CTRL0	Remember that the control signals (START, STOP and RELOAD) can be generated by signals from 16 other peripherals. You can select for each of the three control signals a preferred peripheral from the list of 16.
TCPWM_CNTx_CTRL1	After choosing the source of a control signal, you can configure how your chosen signal can generate the control signal [Rising edge/Falling edge/Both edges]
TCPWM_CNTx_CTRL2	Bits to determine how should Line_Output signal be modified. What should happen to Line_Out when a COMAPARE match event is detected? What should happen to Line_Out when a PERIOD match event is detected?
TCPWM_CNTx_INTRyy	Interrupt management registers. You can find out whether or not is an event active, you can even clear the active event or even mask an event from happening if you prefer!

Clocking system in PSoC



Peripheral Clock Divider Configuration Register PERI_DIV_16_CTLx

Bits	Name	Description
0	ENABLE_x	Divider enabled. HW sets this field to '1' as a result of an ENABLE command. HW sets this field to '0' as a result on a DISABLE command.
23:8	INT16_DIV_x	Integer division by (1+INT16_DIV). Allows for integer divisions in the range [1, 65536].

Clocking system in PSoC

PERI#_CLK	Peripheral
0	SCB0
1	SCB1
2	SCB2
3	SCB3
4	SCB4
5	CSD
6	TCPWM0
7	TCPWM1
8	TCPWM2
9	TCPWM3
10	TCPWM4
11	TCPWM5
12	TCPWM6
13	TCPWM7
14	SmartIO
15	SmartIO
16	SmartIO
17	LCD
18	SAR ADC

General program flow for timers

1)Configure the peripheral clock

- a)Disable the specific divider
- b)Configure the specific divider with divider value
- c)Enable the specific divider
- d)Assign this clock to the required peripheral

```
//TIMER 2 TIMER- CLOCK
SysClk_PeriphDisableDivider(CY_SYSCLK_DIV_16_BIT, 3U);
SysClk_PeriphSetDivider(CY_SYSCLK_DIV_16_BIT, 3U, 24000 - 1);
SysClk_PeriphEnableDivider(CY_SYSCLK_DIV_16_BIT, 3U);
SysClk_PeriphAssignDivider(PCLK_TCPWM_CLOCKS2, CY_SYSCLK_DIV_16_BIT, 3U);
```

2)Set the interrupt priority and enable the IRQ

```
/*TIMER 2 - INTERRUPT*/

NVIC_SetPriority(19, 1u); //timer 2 IRQn = 19, priority 1
NVIC_ClearPendingIRQ(19u); //Clearing and enabling the GPIO interrupt in NVIC
NVIC_EnableIRQ(19u);
```

3)Configure the TCPWM block

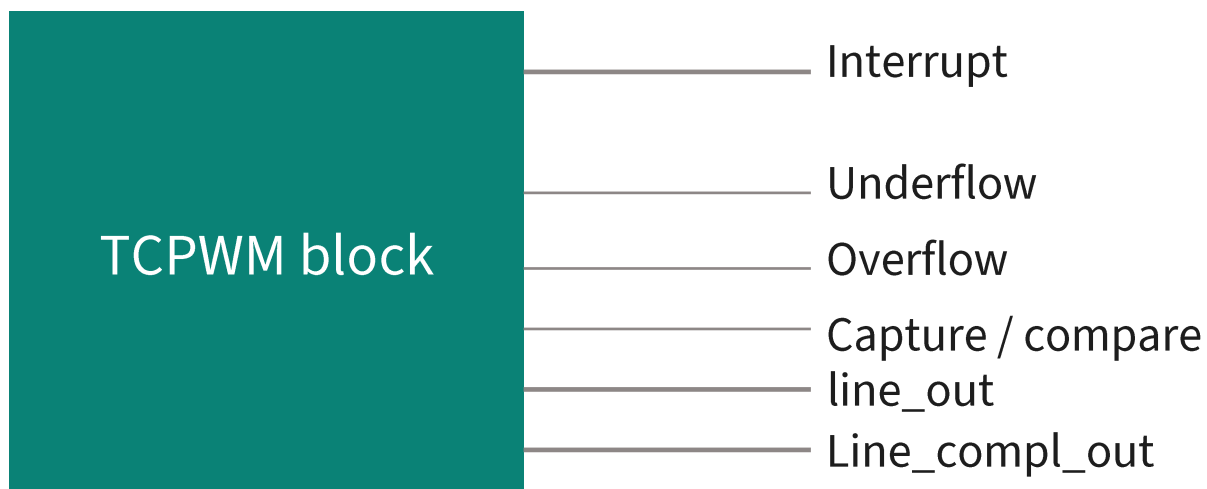
- a)Initialize the TCPWM with configuration like period reg, compare register, mode etc
- b)Enable the timer
- c)Set the type of the interrupt
- d)Trigger the timer

```
/*TIMER 2 - INIT*/
TCPWM_Counter_Init(TCPWM, 2U, &TIMER2_config);
TCPWM_Counter_Enable(TCPWM, 2u);
TCPWM_SetInterrupt(TCPWM, 2u, 1u); //1- TC, 2 - CC, 3 - TC/CC
TCPWM_Counter_SetPeriod(TCPWM, 2u, TIMER_PERIOD_MSEC-1 );
TCPWM_TriggerStart(TCPWM, (1UL << 2u));
```



PWM

TCPWM output signals



Counter generates an **internal underflow (UN)** condition when counting down and the count register reaches zero.

Counter generates an **internal overflow (OV)** condition when counting up and the count register reaches the period value.

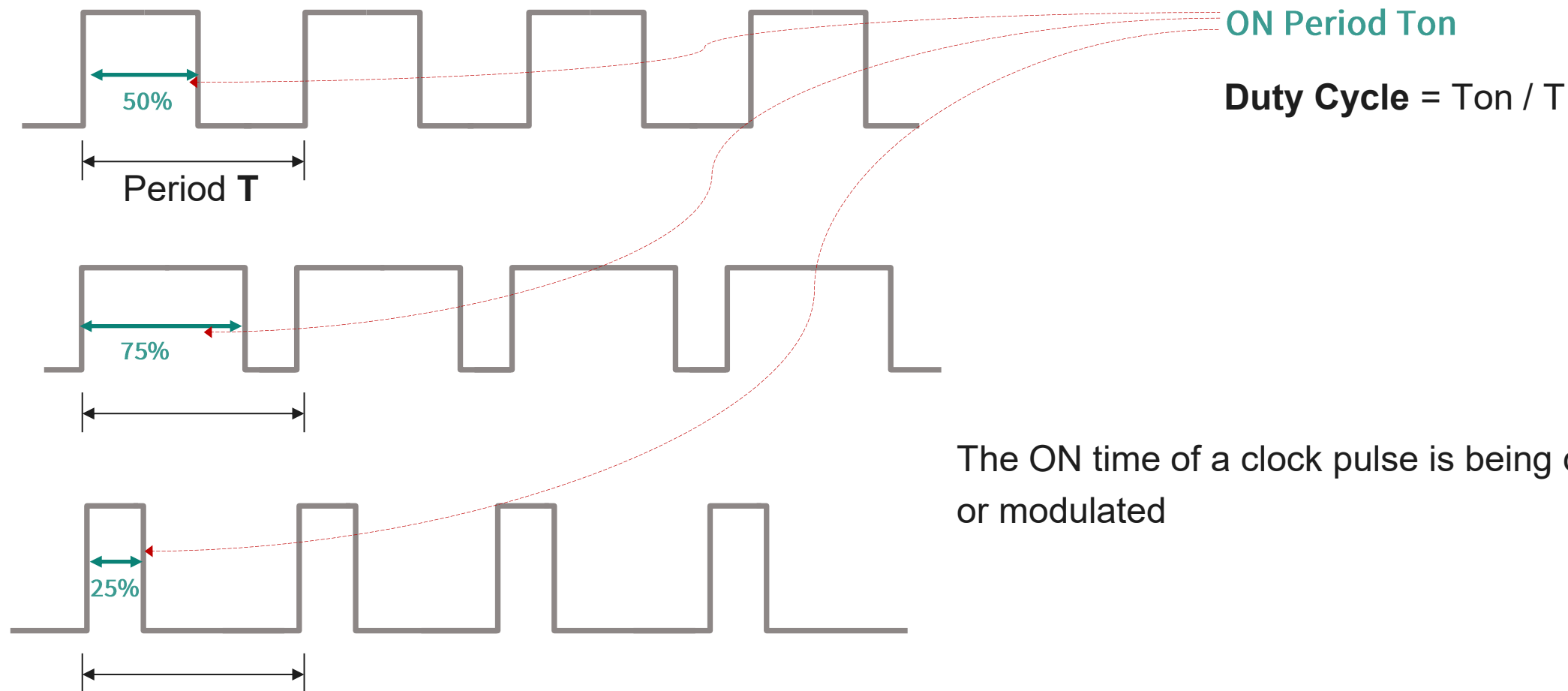
The **capture/compare (CC)** condition is generated by the TCPWM when the counter is running and one of the following conditions occur:

- The counter value equals the compare value.
- When a capture event occurs

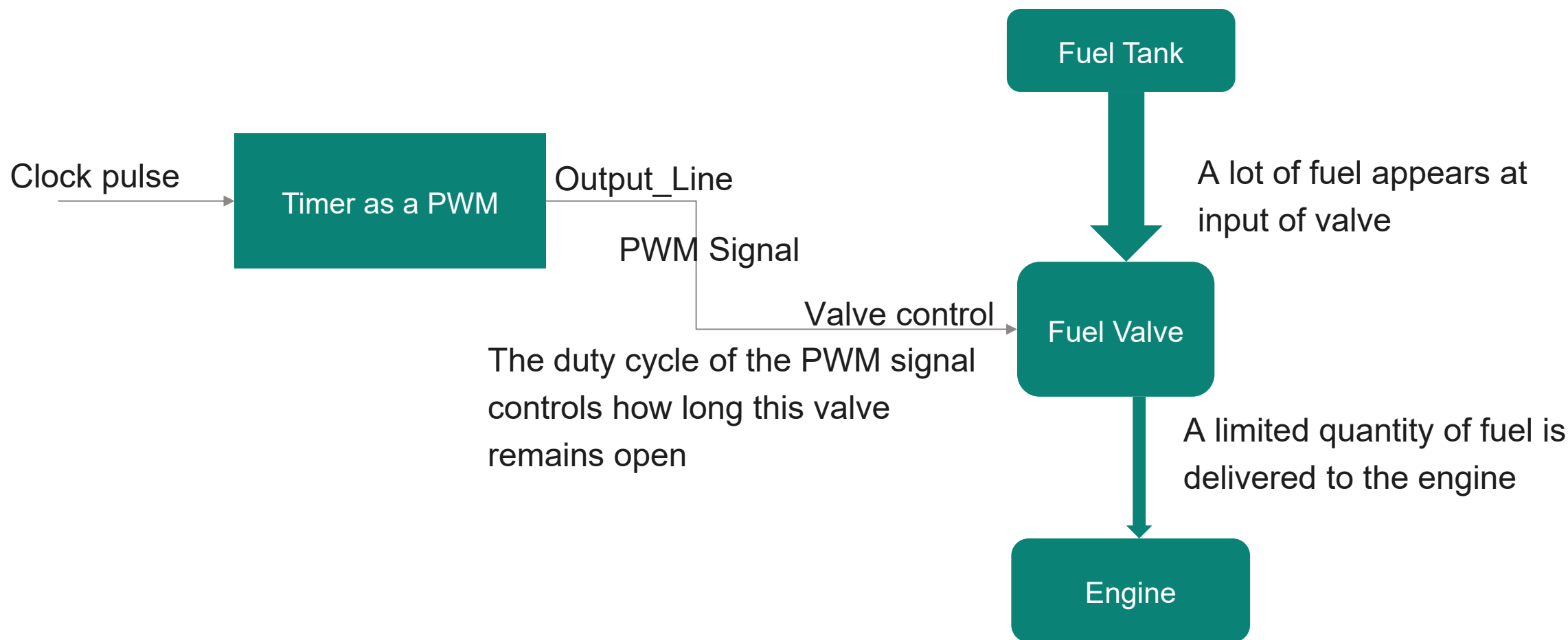
PSoC 4100S Plus have 8 16-bit TCPWM blocks

Hardware timer as PWM (Pulse Width Modulator)

Pulse Width Modulation



Hardware timer as PWM (Pulse Width Modulator) – Example 1



Hardware timer as PWM (Pulse Width Modulator) – Example 2



The duty cycle of the PWM signal controls how bright can this lamp burn

Example:

80% Duty Cycle = Bright

40% Duty Cycle = Normal

15% Duty Cycle = Dim

Hardware timer as Pulse Width Modulator



1. You want to generate a 1 KHz square wave with a duty cycle of 40%.
 Period of 1 KHz square wave = 1 ms
2. Clock frequency of the Timer is 100 KHz (Clock Period: 10 us)
 So Period register is loaded with a value of **100** ($10\text{ us} \times 100 = 1\text{ ms}$)
 $40\% \text{ of } 100 = 40$
 So Compare register is loaded with a value of 40
3. Start the timer
4. Counter starts counting from 0.
5. PWM Output remains 0 until as long as Counter register is less than Compare register
6. Once Counter register equals Compare register, PWM Output flips to HIGH
7. Once Counter register equals Period register, PWM Output resets to 0 and the story continues

How to change PERIOD and COMPARE registers

- If your timer has stopped, you may directly program PERIOD and COMPARE registers
- If the timer is already running with a certain value programmed in PERIOD and COMPARE register
 - Program PERIOD buffer register and COMPARE buffer register
 - Values are automatically transferred from buffer registers to actual registers only after the timer has finished counting up to the value currently programmed in the PERIOD register – Useful in Continuous mode of operation

